RABBIT HOLES

Accelerate · Manipal University Jaipur

16

PHASE

Scale, origins, and the long tail

Where git came from, why that explains the strange parts, and what is left.

Part of **git started**, Accelerate's third workshop.
github.com/accelerate-muj/git-started · MIT licensed.

Contents

1 Ten days in April

In 2005 the Linux kernel used a proprietary version control system called BitKeeper, provided free to the project under a licence that forbade reverse engineering. That arrangement collapsed when the relationship between the vendor and some kernel developers broke down, and the kernel, with over two million lines and thousands of contributors, suddenly had no version control.

Linus Torvalds wrote the first version of git in about ten days. It was self-hosting after a few, and the kernel was using it within weeks.

The requirements he was solving for were specific and unusual:

Distributed Thousands of developers, no central server, most of them offline much of the time and unknown to each other.

Fast Merges and diffs on a two million line tree had to take seconds.

Verifiable Patches arrive from strangers by email. It must be impossible to alter history undetectably.

Branch cheaply Kernel development is many parallel experimental lines.

> **WHY THIS EXISTS**
>
> Almost every part of git that feels strange is a direct answer to one of those.
> Content addressed storage, from Phase 9, is the verifiability requirement. Snapshots rather than diffs are the speed requirement. Branches as forty-one byte files are the cheap branching requirement. The staging area exists so you can assemble a clean patch from messy work before mailing it.
> And authorship being an unverified string, from Phase 10, is the distributed requirement: there is no central authority to check against, so identity is asserted by the author and proven, if at all, by signing.
> Git is not a general purpose tool that happens to suit the kernel. It is a kernel tool that turned out to suit everyone.

2 The email workflow

GitHub did not exist until 2008. For three years git was used the way the kernel still uses it today: patches by email to a mailing list. It is worth seeing, because these commands are still there and still used by some of the most important projects in the world.

```
git format-patch main..feature
git format-patch -3 --cover-letter -o outgoing/
```

This writes one file per commit, named `0001-Add-parser.patch`, containing the diff, the message, the author, and the date.

```
git send-email --to=maintainer@example.com outgoing/*.patch
```

On the other end, the maintainer applies them:

```
git am < 0001-Add-parser.patch
git am --abort
git am --continue
```

`git am` preserves the original author, date and message, so the contributor gets proper credit in the history even though the maintainer ran the command. That is the whole point, and it is why `git apply`, which only applies the diff, is the lesser tool.

> **NOTE**
>
> This explains several conventions that look arbitrary otherwise.
> **Why commit messages have a short subject line and a blank line then a body:** because they became the subject and body of an email.
> **Why `Signed-off-by` exists** and why the kernel requires it: it was a statement of provenance attached to a patch arriving from a stranger, which is Phase 15's DCO.
> **Why a pull request is called a pull request:** the original form was an email asking a maintainer to `git pull` from your public repository. `git request-pull` still generates that message.

```
git request-pull v1.0.0 https://github.com/me/project feature
```

3 Making a large repository fast

Git's performance assumptions break somewhere around the point where the working tree has hundreds of thousands of files. Phase 9 explained why: `git status` compares filesystem metadata for every tracked path.

```
git config core.fsmonitor true
git config core.untrackedCache true
git config feature.manyFiles true
git commit-graph write --reachable
git maintenance start
```

fsmonitor A background process watches the filesystem and tells git which files changed, so `git status` stops walking the tree. The single biggest win on a large repository.

untrackedCache Caches which directories contain untracked files.

commit-graph A precomputed index of the commit history, which speeds up `git log`, merge base calculation, and anything that walks history.

maintenance Schedules the above to stay current, plus background prefetching so your fetches are already mostly done.

3.1 Scalar

Git ships with a tool that turns all of this on together, plus partial clone and sparse checkout from Phase 8:

```
scalar clone https://github.com/big/monorepo.git
scalar register
```

It came out of Microsoft's work getting the Windows source, which is enormous, into git. If you are on a repository big enough to hurt, start here rather than tuning settings individually.

3.2 Monorepos

One repository holding many projects. Google, Meta and Microsoft famously use them.

The appeal: one commit can change a library and every caller atomically, there is one version of everything, and refactoring across project boundaries is possible at all.

The cost: the repository gets huge, CI has to work out what actually needs rebuilding, and standard git tooling strains. Sparse checkout and partial clone are what make it survivable, along with a build system that understands the dependency graph.

> **NOTE**
>
> Almost no student project needs a monorepo, and quite a few adopt one because large companies do. The relevant question is whether your projects genuinely need to change together. If they do not, separate repositories are simpler in every way.

4 The long tail

Real features, less commonly used, each solving one specific problem.

4.1 Bundles

A whole repository, or part of one, in a single file.

```
git bundle create repo.bundle --all
git bundle create recent.bundle main~10..main
git clone repo.bundle my-project
git bundle verify repo.bundle
```

For getting a repository across an air gap, onto a machine with no network, or onto a USB stick. It is a complete, verifiable transfer with no server involved, which is the distributed design showing through.

4.2 Notes

Attach information to a commit without changing it.

```
git notes add -m "Reviewed by the security team" <sha>
git notes show <sha>
git log --show-notes
git push origin refs/notes/commits
```

Because notes live in a separate ref, adding one does not alter the commit or its hash. Used for review metadata, test results, and build provenance. They must be pushed explicitly, which is why most people have never seen one.

4.3 Archive

```
git archive --format=zip --output=release.zip v1.0.0
git archive --format=tar.gz --prefix=project/ HEAD > project.tar.gz
```

Exports a tree without `.git`. Respects `export-ignore` from `.gitattributes`, so you can keep tests and CI config out of a source tarball.

4.4 Orphan branches

A branch with no history at all:

```
git switch --orphan gh-pages
git rm -rf .
```

The classic use is a documentation branch that shares no ancestry with the code. This is what `gh-pages` originally was.

4.5 Unrelated histories

```
git merge --allow-unrelated-histories other/main
```

Git refuses to merge two histories with no common ancestor, because it is almost always a mistake. When you are genuinely combining two separate projects, this is the override.

4.6 Empty commits

```
git commit --allow-empty -m "Trigger a rebuild"
```

Useful for kicking CI. `gh run rerun` is usually better, but this works when nothing else will.

4.7 Replace

```
git replace <old-sha> <new-sha>
```

Makes git substitute one object for another when reading history, without rewriting anything. Used to graft a converted historical archive onto the front of a repository without changing any modern hashes.

4.8 Migrating from elsewhere

```
git svn clone https://svn.example.com/repo --stdlayout
```

Preserves history from Subversion. Mercurial has `hg-fast-export`. Both are one time operations, so accept a slow, careful run over a fast, wrong one.

5 What to read next

The Pro Git book Free, online, and genuinely the reference. Chapter 10 is the plumbing material from Phase 9 in more depth.

git help <command> Every command has a manual page, and unlike most software they are good. `git help revisions` and `git help everyday` are worth an hour on their own.

The git source Written in C, still readable, and the original design documents are in `Documentation/technical/`.

Your own repositories The one that matters. Nothing here is learned by reading it.

6 The end of the rabbit hole

Thirteen phases beyond the workshop, and the useful summary is short.

Git stores snapshots, addressed by the hash of their contents, linked into a graph by parent pointers. Branches are labels on that graph. Everything else, every command in this handbook, is a way of adding to the graph, moving a label, or reading what is already there.

Once that sentence is genuinely obvious rather than memorised, you no longer need this document, which is the point of it.

> **CHECK YOURSELF**
>
> 1. Name three design decisions in git that come directly from the kernel's requirements in 2005.
> 2. Why does a commit message have a subject line, a blank line, then a body?
> 3. What does `git am` preserve that `git apply` does not?
> 4. Why is a pull request called a *pull* request?
> 5. Which setting gives the biggest `git status` speedup on a large repository, and why does it work?
> 6. Explain git to somebody in three sentences, without using the word *version*.

That is all of it.

You started by making a folder and running `git init`. You have now been through undo, rewriting history, forensics, merge strategies, remotes, the object database, signing, configuration, GitHub, the CLI, CI, maintainership, and the reason any of it is shaped the way it is.

Go and break something. You know how to get it back.

Corrections and additions: github.com/accelerate-muj/git-started